30.2. Python: SciPy

Scientific Computation using SciPy

Previous Section:

</> 30.1. Python: NumPy

Contents:

NumPy provides the foundation for numerical computation, but eventually we encounter problems that go beyond simple arrays and matrices. We may need advanced statistical analysis, probability distributions, optimization techniques, numerical integration, graph algorithms, or memory-efficient representations of massive datasets.

This is where **SciPy** becomes useful.

SciPy (Scientific Python) is an open-source scientific computing library built on top of NumPy. While NumPy focuses primarily on arrays and matrix operations, SciPy extends those capabilities with specialized algorithms commonly used in mathematics, engineering, data science, machine learning, and scientific research.

```
import scipy as sp
```

Just as `np` became the common alias for NumPy, many programmers use `sp` when working with SciPy.

2. Common Operations on SciPy

2.1. Sparse Matrices

In many real-world applications, most values inside a matrix are actually zeros. For example: Social network connections; Recommendation systems; Graph representations; Text processing; Scientific simulations

Storing millions of zeros wastes memory and computation time.

A **Sparse Matrix** is a matrix in which most values are zero. Instead of storing every element, SciPy stores only the non-zero values, greatly reducing memory usage and often improving performance.

A matrix is generally considered sparse when the number of zeros exceeds the number of non-zero entries.

- Dense Matrix vs Sparse Matrix: A normal NumPy matrix stores everything.

```
import numpy as np

dense = np.array([[0, 1, 0],
                  [0, 0, 2],
                  [3, 0, 4]])

print("Dense Matrix:\n", dense)
```

```
Dense Matrix:
[[0 1 0]
[0 0 2]
[3 0 4]]
```

SciPy can convert it into a sparse representation:

```
from scipy.sparse import coo_matrix
# Others are csr_matrix and csc_matrix

sparse = coo_matrix(dense)
print(sparse)
```

```
<COOrdinate sparse matrix of dtype 'int64'
with 4 stored elements and shape (3, 3)>
Coords  Values
(0, 1)  1
(1, 2)  2
(2, 0)  3
(2, 2)  4
```

Only the non-zero values are stored internally. The zeros still conceptually exist in the matrix, but they are not occupying storage space.

SciPy supports several sparse formats:

Format	Description
COO	Coordinate Format
CSR	Compressed Sparse Row
CSC	Compressed Sparse Column

The details of their storage mechanisms are usually less important than understanding their purpose:

- COO is easy to construct.
- CSR is efficient for row operations.
- CSC is efficient for column operations.

Useful Methods on a sparse matrix

- Count all non-zero entries:

```
print(sparse.count_nonzero())
```

```
4
```

- Count non-zeros by row or column:

```
print(sparse.getnnz())
print(sparse.getnnz(axis=0))   # columns
print(sparse.getnnz(axis=1))   # rows
```

```
4
[1 1 2]
[1 1 2]
```

- Matrix shape:

```
print(sparse.shape)
```

```
(3, 3)
```

- Stored data size:

```
print(sparse.data.shape)
```

```
(4, )
```

These methods are useful because sparse matrices only store non-zero values internally.

- Converting Back to NumPy: Sometimes we need the original dense matrix again.

```
dense_again = sparse.toarray()
print("Dense Matrix (revised):\n", dense_again)
```

```
Dense Matrix (revised):
[[0 1 0]
[0 0 2]
[3 0 4]]
```

This reconstructs all the missing zeros and returns a standard NumPy array.

2.2. Statistical Operations

Statistics is one of SciPy's strongest features.

The module below:

```
import scipy.stats as stats
```

contains powerful tools for descriptive statistics, hypothesis testing, probability distributions, and data analysis.

- Descriptive Statistics: Display a quick summary of a dataset

```
import scipy.stats as stats

data = [1, 2, 2, 3, 4, 5, 6, 8]

result = stats.describe(data)
print(result)
```

DescribeResult(nobs=np.int64(8), minmax=(np.int64(1), np.int64(8)), mean=np.float64(3.875), variance=np.float64(5.553571428571428), skewness=np.float64(0.5087022045579557), kurtosis=np.float64(-0.855305466237942))

Output contains: Number of observations; Minimum and maximum; Mean; Variance; Skewness; Kurtosis, all computed in a single function call.

- Mode: The mode is the most frequently occurring value.

```
print(stats.mode(data))
```

ModeResult(mode=np.int64(2), count=np.int64(2))
→ For [1, 2, 2, 3, 4], the mode is 2

- Trimmed Statistics: Outliers can distort statistical summaries. SciPy provides trimmed statistics that ignore extreme values.

```
print(stats.tmean(data))
print(stats.tvar(data))
print(stats.tstd(data))
print(stats.tmin(data))
print(stats.tmax(data))
```

```
3.875
5.553571428571428
2.3566016694748027
1
8
```

These functions help reduce the influence of unusually large or small observations.

- Skewness measures asymmetry.

```
print(stats.skew(data))
```

```
0.5087022045579557
```

Interpretation:

Value	Meaning
> 0	Right-skewed
< 0	Left-skewed
= 0	Symmetric

A positive skew means the data has a longer tail on the right side.

- Kurtosis measures the presence of extreme values (outliers).

```
print(stats.kurtosis(data))
```

```
-0.855305466237942
```

Interpretation:

Value	Meaning
> 0	More outliers than normal
< 0	Fewer outliers than normal
= 0	Similar to normal distribution

Think of kurtosis as measuring how "heavy" the tails of a distribution are.

2.3. Probability Distributions

Probability distributions describe how random events behave. SciPy provides implementations for many classical distributions.

- Bernoulli Distribution: A single trial with only two outcomes, which are success or failure. Examples are coin flip, light bulb works or fails, student passes or fails

```
from scipy.stats import bernoulli

dist = bernoulli(0.7)

# Probability of success
print(dist.pmf(1))

# Probability of failure
print(dist.pmf(0))
```

```
0.7
0.30000000000000004
```

- Binomial Distribution: Multiple Bernoulli trials. Examples include tossing a coin 10 times, or counting the number of heads.

```
from scipy.stats import binom

dist = binom(10, 0.5)

# Probability of exactly 6 heads
print(dist.pmf(6))
```

- **Poisson Distribution:** Used when counting events over time or space. Examples include customers entering a store per hour, emails arriving per minute, website visits per second

```
from scipy.stats import poisson

dist = poisson(mu=5)

# Probability of exactly 3 events
print(dist.pmf(3))
```

```
0.1403738958142805
```

- **Normal Distribution:** The famous bell curve. Many natural measurements approximately follow a normal distribution such as heights, IQ scores, measurement errors, exam scores

```
from scipy.stats import norm

dist = norm(loc=0, scale=1)

# Probability density at x = 1
print(dist.pdf(1))
```

```
0.24197072451914337
```

- **Common Distribution Methods**
  - `dist.pmf(k)`: Probability of exactly k events . Used for discrete distributions.
  - `dist.cdf(k)`: Probability of observing at most k events. Mathematically: $P(X \leq k)$
  - `dist.sf(k)`: Probability of exceeding k. Equivalent to: $P(X > k)$
  - `dist.pdf(x)` : Returns the probability density at a given location, commonly used for continuous distributions.
  - `dist.mean(); dist.var(); dist.std()`: Respectively mean, variance, and standard deviation. These methods work for nearly every SciPy probability distribution.

2.4. Numerical Methods

Many scientific and engineering problems cannot be solved exactly using algebra alone. Instead, computers use **Numerical Methods** to approximate solutions. SciPy provides a collection of highly optimized numerical algorithms, allowing us to solve problems that would otherwise require implementing complex mathematical procedures manually.

This section focuses primarily on how to use the functions. The mathematical theory behind these methods will be covered in later courses.

- Z-Score: A **Z-Score** measures how many standard deviations a value is from the mean. It is commonly used for outlier detection, data normalization, and machine Learning preprocessing

```
import scipy.stats as stats

data = [10, 15, 20, 25, 30]

z_scores = stats.zscore(data)
print(z_scores)
```

[-1.41 -0.71 0.00 0.71 1.41]

Interpretation:

- Positive value → above the mean
- Negative value → below the mean
- Large absolute values (typically > 3) may indicate outliers.

- Linear Regression: Linear Regression fits a line: $Y = A + BX$ to a collection of data points.

```
from scipy import stats

x = [1, 2, 3, 4, 5]
y = [3, 5, 7, 9, 11]

result = stats.linregress(x, y)

print("Slope:", result.slope)
print("Intercept:", result.intercept)
```

Slope: 2.0
Intercept: 1.0
Meaning: Y = 1 + 2X

This technique is commonly used in forecasting, prediction, and machine learning.

- Random Sampling: Most probability distributions can generate random samples directly.

```
from scipy.stats import norm

samples = norm.rvs(loc=0, scale=1, size=5)
print(samples)
```

Possible output: [1.08962341 0.77692256 -1.49665594 0.05424878 -2.75140426]

This is useful for simulations, monte Carlo methods, synthetic dataset generation

- Numerical Differentiation: Derivatives measure how quickly a function changes. SciPy can approximate derivatives numerically.

```
from scipy.differentiate import derivative

f = lambda x: x**2
print(derivative(f, 3))
```

Expected result: 6
Because: d/dx (x^2) = 2x → 2(3) = 6

Numerical differentiation is useful when analytical differentiation is difficult or impossible.

- Numerical Integration: Integration finds area under a curve for a known function.

```
from scipy.integrate import quad

f = lambda x: x**2

result, error = quad(f, 0, 2)
print(result)
```

2.666666666666667

This computes: $\int_0^2 x^2 dx$ without requiring manual integration.

- Numerical Integration from Data: Sometimes we only have data points instead of a function.

```
import numpy as np
from scipy.integrate import trapezoid

x = np.array([0,1,2,3])
y = np.array([0,1,4,9])

area = trapezoid(y, x)
print(area)
```

9.5

SciPy also provides `simpson()` and `romb()`, which use different numerical integration techniques.

- Initial Value Problems (ODEs): Many physical systems are described by Ordinary Differential Equations (ODEs). Examples are population growth, radioactive decay, planetary motion, or electrical circuits. SciPy provides `solve_ivp()`.

```
from scipy.integrate import solve_ivp

def f(t, y):
    return y

result = solve_ivp(f, [0, 2], [1])
print(result.y)
```

```
[[1.         1.10519301 2.9040598  7.38932547]]
```

This solves: $\frac{dy}{dt} = y, \quad y(0) = 1$ numerically.

- Minimization and Maximization: Optimization is the process of finding the best solution. For example, finding the minimum value of $f(x) = x^2 + 4$

```
from scipy.optimize import minimize_scalar

f = lambda x: x**2 + 4

result = minimize_scalar(f)

print(result.x)
print(result.fun)
```

1.1473103082619 37e-08
4.0

Meaning minimum occurs at $x = 0$ (or a value small enough it is near 0) and the minimum value is 4.

SciPy provides `minimize_scalar()` and `minimize()` for one-variable and multi-variable optimization problems.

2.5. Working with Graphs

A **Graph** is one of the most important data structures in Computer Science. A graph consists of: **Vertices (Nodes)** — Objects and **Edges** — Connections between objects

Examples include: Social networks; Road maps; Internet routing; Recommendation systems; Web page links

SciPy represents graphs using **Sparse Matrices (or Adjacency Matrix)**, making graph computations memory-efficient. In the matrix, rows represent source vertices. Columns represent destination vertices.

```
from scipy.sparse.csgraph import csgraph_from_dense
```

- **Directed and Undirected Graphs**

A graph can be **directed when** $A \to B$, in which direction matters. Or undirected when $A — B$, in which connections work both ways.

```
# Undirected Graph
undirected_graph = np.array([[0, 1, 1, 0],
                             [1, 0, 1, 1],
                             [1, 1, 0, 1],
                             [0, 1, 1, 0]])
# Directed Graph
directed_graph = np.array([[0, 1, 1, 0],
                           [0, 0, 1, 1],
                           [0, 0, 0, 1],
                           [0, 0, 0, 1]])

# Represent the graphs as sparse matrices
undirected_sparse = csgraph_from_dense(undirected_graph)
directed_sparse = csgraph_from_dense(directed_graph)
```

- **Weighted and Unweighted Graphs**

An unweighted graph $A \rightarrow B$ means every edge has equal cost. While a weighted graph $A --5--> B$ means each edge stores a weight such as distance, cost, or time.

```
# Unweighted Graphs
unweighted_graph = np.array([[0, 1, 1, 0],
                             [0, 1, 1, 1],
                              [1, 0, 0, 0],
                               [0, 1, 0, 0]])

# Represent the graph as sparse matrices
unweighted_sparse = csgraph_from_dense(unweighted_graph)
```

```
# Weighted Graph
weighted_graph = np.array([[0, 4, 3, 0],
                           [0, 5, 6, 2],
                           [3, 0, 0, 0],
                           [0, 4, 0, 0]])
```

```
# Represent the graph as sparse matrices
weighted_sparse = csgraph_from_dense(weighted_graph)
```

- **Connected Components**

Connected components identify groups of connected vertices. SciPy supports weakly connected components and strongly connected components

```
# Connected Components
from scipy.sparse.csgraph import connected_components

connected_graph = np.array([[0, 0, 1, 0, 0],
                            [0, 0, 0, 1, 1],
                             [1, 0, 0, 0, 0],
                              [0, 0, 0, 0, 0],
                               [0, 0, 0, 0, 0]])

# Convert to sparse matrix
sparse_matrix = coo_matrix(connected_graph)

# Weakly connected components
weak_components, labels = connected_components(csgraph=spar
se_matrix,
                                               directed=Tru
e, connection='weak')
print("Num of Weak Components:", weak_components)
print("Labels:", labels)

# Strongly connected components
strong_components, labels = connected_components(csgraph=sp
arse_matrix,
                                                 directed=
True, connection='strong')
print("Num of Strong Components:", strong_components)
print("Labels:", labels)
```

Weak connectivity ignores edge directions. Strong connectivity requires paths in both directions.

- **Shortest Path**

Finding the shortest path is one of the most famous graph problems.

```
from scipy.sparse.csgraph import shortest_path
from scipy.sparse import csr_matrix

graph = np.array([[0, 4, 5, 0, 0, 0],
                  [4, 0, 11, 9, 7, 0],
                   [5, 11, 0, 0, 3, 0],
                    [0, 9, 0, 0, 13, 2],
                     [0, 7, 3, 13, 0, 6],
                      [0, 0, 0, 2, 6, 0]])

# Convert to sparse matrix
sparse_matrix = csr_matrix(graph)

# Finding shortest path
dist_matrix, predecessors = shortest_path(csgraph=sparse_ma
trix,
                                          directed=False,
                                          return_predecesso
rs=True)
print("Distance Matrix:\n", dist_matrix)
print("Predecessor Matrix:\n", predecessors)
```

```
Distance Matrix:
[[ 0.  4.  5. 13.  8. 14.]
 [ 4.  0.  9.  9.  7. 11.]
 [ 5.  9.  0. 11.  3.  9.]
 [13.  9. 11.  0.  8.  2.]
 [ 8.  7.  3.  8.  0.  6.]
 [14. 11.  9.  2.  6.  0.]]
Predecessor Matrix:
[[-9999     0     0     1     2     4]
 [    1 -9999     0     1     1     3]
 [    2     0 -9999     5     2     4]
 [    1     3     4 -9999     5     3]
 [    2     4     4     5 -9999     4]
 [    2     3     4     5     5 -9999]]
```

This computes the shortest distance between every pair of vertices.

SciPy also exposes several classical algorithms:

```
from scipy.sparse.csgraph import dijkstra, floyd_warshall,
bellman_ford
```

Although they solve the same problem, they use different strategies internally.

- Depth-First Search (DFS) explores as deeply as possible before backtracking.

```
# Depth-First Search
from scipy.sparse.csgraph import depth_first_order, depth_f
irst_tree
from scipy.sparse import csr_matrix

graph = np.array([[0, 1, 1, 1, 0],
                  [1, 0, 1, 0, 0],
                   [1, 1, 0, 0, 1],
                    [1, 0, 0, 0, 0],
                     [0, 0, 1, 0, 0]])

# Convert to sparse matrix
sparse_matrix = csr_matrix(graph)

dfs_order = depth_first_order(csgraph=sparse_matrix, i_star
t=0,
                              directed=False)

print("DFS Order:", dfs_order[0])

dfs_tree = depth_first_tree(csgraph=sparse_matrix, i_start=
0,
                            directed=False)

print("DFS Tree:\n", dfs_tree.toarray())
```

```
DFS Order: [0 1 2 4 3]
DFS Tree:
[[0. 1. 0. 1. 0.]
[0. 0. 1. 0. 0.]
[0. 0. 0. 0. 1.]
[0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0.]]
```

DFS is commonly used for cycle detection, topological sorting, and graph traversal

- Breadth-First Search (BFS) explores level by level.

```
# Breadth-First Search
from scipy.sparse.csgraph import breadth_first_order, bread
th_first_tree
from scipy.sparse import csr_matrix

bfs_order = breadth_first_order(csgraph=sparse_matrix, i_st
art=0,
                                directed=False)

print("BFS Order:", bfs_order[0])

bfs_tree = breadth_first_tree(csgraph=sparse_matrix, i_star
t=0,
                              directed=False)

print("BFS Tree:\n", bfs_tree.toarray())
```

```
BFS Order: [0 1 2 3 4]
BFS Tree:
[[0. 1. 1. 1. 0.]
[0. 0. 0. 0. 0.]
[0. 0. 0. 0. 1.]
[0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0.]]
```

BFS is commonly used for shortest path in unweighted graphs, network exploration, and route finding

Many advanced Computer Science topics can be represented as graphs:

- Google Search rankings
- Social media relationships
- Transportation systems
- Communication networks
- Recommendation engines
- Artificial Intelligence search problems

SciPy's graph tools allow these problems to be solved efficiently while leveraging the memory benefits of sparse matrices.

A Quick Note on NumPy, SciPy, and Math
If you have used Python's built-in `math` module, you may notice that some functions in **NumPy**, **SciPy**, and **Math** appear to be identical. For example:

```
import math
import numpy as np

math.sqrt(16)
np.sqrt(16)
```

Both return: 4

Similarly:

```
math.exp(1)
np.exp(1)

math.log(10)
np.log(10)
```

Each pair produces the same mathematical results.

However, there is an important difference:

- **Math** is designed primarily for single numerical values.
- **NumPy** is designed for arrays and matrices.
- **SciPy** builds upon NumPy and provides more advanced scientific algorithms.

For example:

```python
import numpy as np

arr = np.array([1, 4, 9, 16])

print(np.sqrt(arr))
```

```
[1. 2. 3. 4.]
```

The `math.sqrt()` function cannot directly process an entire NumPy array.
As a rule of thumb:

- Use **Math** for simple calculations on individual values.
- Use **NumPy** when working with arrays, matrices, and numerical datasets.
- Use **SciPy** when you need advanced scientific, statistical, optimization, or numerical methods.
- And additionally, use **SymPy** when you want exact symbolic mathematics rather than numerical approximations.

Although many functions share similar names and behaviors, they are designed for different levels of mathematical computation.

Summary

SciPy extends NumPy with advanced scientific algorithms.

It specializes in:

- Sparse matrices
- Statistics
- Probability distributions
- Numerical methods
- Optimization
- Differential equations

- Graph algorithms

Think of SciPy as the toolbox that solves scientific and engineering problems.

References

https://docs.scipy.org/doc/scipy/

https://www.geeksforgeeks.org/machine-learning/scipy-tutorial/

The next section covers the SymPy library:

</> 30.3. Python: SymPy